
Swings:

AWT is used for creating GUI in Java. However, the AWT components are internally depends on native methods like C functions and operating system equivalent and hence problems related to portability arise (look and feel. Ex. Windows window and MAC window). And, also AWT components are heavy weight. It means AWT components take more system resources like memory and processor time.

Due to this, Java soft people felt it is better to redevelop AWT package without internally taking the help of native methods. Hence all the classes of AWT are extended to form new classes and a new class library is created. This library is called JFC (Java Foundation Classes).

Java Foundation Classes (JFC):

JFC is an extension of original AWT. It contains classes that are completely portable, since the entire JFC is developed in pure Java. Some of the features of JFC are:

1. JFC components are light-weight: Means they utilize minimum resources.
2. JFC components have same look and feel on all platforms. Once a component is created, it looks same on any OS.
3. JFC offers “pluggable look and feel” feature, which allows the programmer to change look and feel as suited for platform. For, ex if the programmer wants to display window-style button on Windows OS, and Unix style buttons on Unix, it is possible.
4. JFC does not replace AWT, but JFC is an extension to AWT. All the classes of JFC are derived from AWT and hence all the methods in AWT are also applicable in JFC.

So, JFC represents class library developed in pure Java which is an extension to AWT and swing is one package in JFC, which helps to develop GUIs and the name of the package is

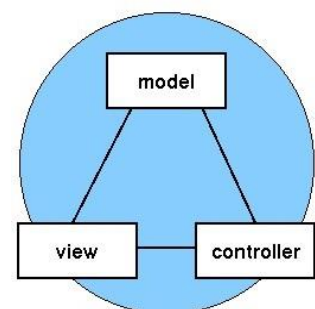
```
import javax.swing.*;
```

Here x represents that it is an ‘extended package’ whose classes are derived from AWT package.

MVC Architecture:

In MVC terminology,

- Model corresponds to the state information associated with the component (data).
For example, in the case of a check box, the model contains a field that indicates if the box is checked or unchecked.
- The view visual appearance of the component based upon model data.



- The controller acts as an interface between view and model. It intercepts all the requests i.e. receives input and commands to Model / View to change accordingly.

Although the MVC architecture and the principles behind it are conceptually sound, the high level of separation between the view and the controller is not beneficial for Swing components. Instead, Swing uses a modified version of MVC that combines the view and the controller into a single logical entity called the UI delegate. For this reason, Swing's approach is called either the Model-Delegate architecture or the Separable Model architecture.

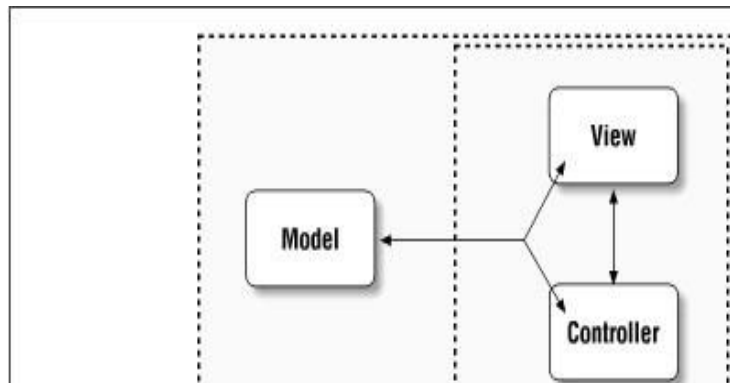


Figure : With Swing, the view and the controller are combined into a UI-delegate object

So let's review: each Swing component contains a model and a UI delegate. The model is responsible for maintaining information about the component's state. The UI delegate is responsible for maintaining information about how to draw the component on the screen. In addition, the UI delegate reacts to various events.

Difference between AWT and Swings:

AWT	Swing
Heavy weight	Light weight
Look and feel is OS based	Look and feel is OS independent.
Not pure Java based	Pure Java based
Applet portability: Web-browser support	Applet portability: A plug-in is required
Do not support features like icon and tool tip.	It supports.
The default layout manager for applet: flow and frame is border layout.	The default layout manger for content pane is border layout.

Components and Containers:

A Swing GUI consists of two key items: **components** and **containers**.

However, this distinction is mostly conceptual because all containers are also components. The difference between the two is found in their intended purpose: As the term is commonly used, a *component* is an independent visual control, such as a push button or slider. A container holds a group of components. Thus, a container is a special type of component that is designed to hold other components.

Furthermore, in order for a component to be displayed, it must be held within a container. Thus, all Swing GUIs will have at least one container. Because containers are components, **a container can also hold other containers**. This enables Swing to define what is called a *containment hierarchy*, at the top of which must be a *top-level container*.

Components:

In general, Swing components are derived from the **JComponent** class. **JComponent** provides the functionality that is common to all components. For example, **JComponent** supports the pluggable look and feel. **JComponent** inherits the AWT classes **Container** and **Component**. All of Swing's components are represented by classes defined within the package **javax.swing**. The following figure shows hierarchy of classes of javax.swing.

JApplet (Deprecated)	JButton	JCheckBox	JCheckBoxMenuItem
JColorChooser	JComboBox	JComponent	JDesktopPane
JDialog	JEditorPane	JFileChooser	JFormattedTextField
JFrame	JInternalFrame	JLabel	JLayer
JLayeredPane	JList	JMenu	JMenuBar
JMenuItem	JOptionPane	JPanel	JPasswordField
JPopupMenu	JProgressBar	JRadioButton	JRadioButtonMenuItem
JRootPane	JScrollBar	JScrollPane	JSeparator
JSlider	JSpinner	JSplitPane	JTabbedPane
JTable	JTextArea	JTextField	JTextPane
JToggleButton	JToolBar	JToolTip	JTree
JViewport	JWindow		

Containers:

Swing defines two types of containers.

1. **Top-level containers/ Root containers:** JFrame, JApplet, JWindow, and JDialog.

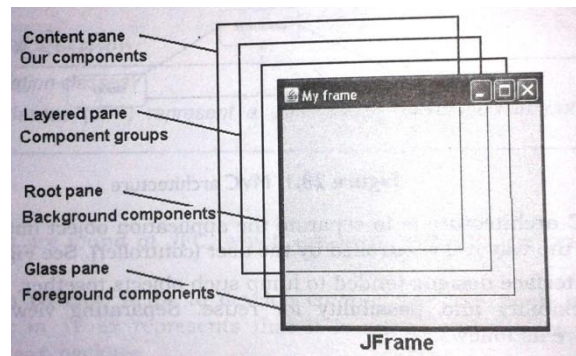
As the name implies, a top-level container must be at the top of a containment hierarchy. A top-level container is not contained within any other container.

Furthermore, every containment hierarchy must begin with a top-level container. The one most commonly used for applications are JFrame and JApplet.

Unlike Swing's other components, the top-level containers are heavyweight. Because they inherit AWT classes Component and Container.

Whenever we create a top level container four sub-level containers are automatically created:

- Glass pane (JGlass)
- Root pane (JRootPane)
- Layered pane (JLayeredPane)
- Content pane



Glass pane: This is the first pane and is very close to the monitor's screen. Any components to be displayed in the foreground are attached to this glass pane. To reach this glass pane we use `getGlassPane()` method of JFrame class, which return Component class object.

Root Pane: This pane is below the glass pane. Any components to be displayed in the background are displayed in this frame. To go to the root pane, we can use `getRootPane()` method of JFrame class, which returns JRootPane object.

Layered pane: This pane is below the root pane. When we want to take several components as a group, we attach them in the layered pane. We can reach this pane by calling `getLayeredPane()` method of JFrame class which returns JLayeredPane class object.

Content pane: This is bottom most of all, Individual components are attached to this pane. To reach this pane, we can call `getContentPane()` method of JFrame class which returns Container class object.

2. **Lightweight containers** – containers do inherit JComponent. An example of a lightweight container is JPanel, which is a general-purpose container. Lightweight containers are often used to organize and manage groups of related components.

JFrame:

Frame represents a window with a title bar and borders. Frame becomes the basis for creating the GUIs for an application because all the components go into the frame.

To create a frame, we have to create an object to JFrame class in swing as

```
JFrame jf=new JFrame(); // create a frame without title
```

```
JFrame jf=new JFrame("title"); // create a frame with title
```

To close the frame, use setDefaultCloseOperation() method of JFrame class

```
setDefaultCloseOperation(constant)
```

where constant values are

JFrame.EXIT_ON_CLOSE	This closes the application upon clicking the close button
JFrame.DISPOSE_ON_CLOSE	This closes the application upon clicking the close button
JFrame.DO_NOTHING_ON_CLOSE	This will not perform any operation upon clicking close button
JFrame.HIDE_ON_CLOSE	This hides the frame upon clicking close button

Example:

```
import javax.swing.*;
class FrameDemo
{
    public static void main(String arg[])
    {
        JFrame jf=new JFrame("SNPSU");
        jf.setSize(200,200);
        jf.setVisible(true);
        jf setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE );
    }
}
```

Example: To set the background

```
import javax.swing.*;
import java.awt.*;
class FrameDemo
{
    public static void main(String arg[])
    {
        JFrame jf=new JFrame("SNPSU");
        jf.setSize(200,200);
        jf.setVisible(true);
        Container c=jf.getContentPane();
        c.setBackground(Color.green);
    }
}
```

JLabel:

- JLabel is used to display a text
 - JLabel(string str)
 - JLabel(Icon i)
 - JLabel(String s, Icon i, int align)
 - CENTER, LEFT, RIGHT, LEADING, TRAILING
- Icon – is an interface
 - The easiest way to obtain icon is to use ImageIcon class. ImageIcon class implements Icon interface.

Important Methods:

Icon getIcon()

String getText()

void setIcon(Icon icon)

void setText(String s)

JText Fields

The Swing text field is encapsulated by the **JTextComponent** class, which extends **JComponent**. It provides functionality that is common to Swing text components. One of its subclasses is **JTextField**, which allows you to edit one line of text. Some of its constructors are shown here:

```
JTextField( )  
JTextField(int cols)  
JTextField(String s, int cols)  
JTextField(String s)
```

Here, *s* is the string to be presented, and *cols* is the number of columns in the text field.

The following example illustrates how to create a text field. The applet begins by getting its content pane, and then a flow layout is assigned as its layout manager. Next, a **JTextField** object is created and is added to the content pane.

Example:

```
import javax.swing.*;  
import java.awt.*;  
import java.awt.event.*;  
class MyFrame extends JFrame implements ActionListener  
{  
    JLabel jl, jl2;  
    JTextField jtf;  
    MyFrame()  
    {
```

```

        setLayout(new FlowLayout());
        jl=new JLabel("Enter your name");
        jl2=new JLabel();
        jtf=new JTextField("SNPSU",15);

        add(jl);
        add(jtf);
        add(jl2);
        jtf.addActionListener(this);
    }
    public void actionPerformed(ActionEvent ae)
    {
        jl2.setText(jtf.getText());
    }
}
class FrameDemo
{
    public static void main(String arg[])
    {
        MyFrame f=new MyFrame();
        f.setTitle("Welcome to Swings");
        f.setSize(500,500);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

Buttons:

Swing defines four types of buttons: **JButton**, **JToggleButton**, **JCheckBox**, and **JRadioButton**. All are subclasses of the **AbstractButton** class, which extends **JComponent**. Thus, all buttons share a set of common traits.

The JButton Class

The **JButton** class provides the functionality of a push button. **JButton** allows an icon, a string, or both to be associated with the push button. Some of its constructors are shown here:

```

JButton(Icon          i)
JButton(String        s)
JButton(String s, Icon i)

```

Here, *s* and *i* are the string and icon used for the button.

Example:

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MyFrame extends JFrame implements ActionListener

```

```

{
    JButton jb,jb1,jb2;
    JLabel jl;
    MyFrame()
    {
        setLayout(new
        FlowLayout());      jl=new
        JLabel();

        jb=new JButton("SCHOOL OF ENGINEERING");
        ImageIcon ii=new ImageIcon("snps.JPG");
        jb1=new JButton("SNPSU",ii);

        ImageIcon ii2=new ImageIcon("cs.JPG");
        jb2=new JButton("CSE", ii2);

        add(jb); add(jb1); add(jb2); add(jl);

        jb.addActionListener(this);
        jb1.addActionListener(this);
        jb2.addActionListener(this);
    }
    public void actionPerformed(ActionEvent ae)
    {
        jl.setText("You Pressed: "+ae.getActionCommand());
    }
}
class FrameDemo
{
    public static void main(String arg[])
    {
        MyFrame f=new MyFrame();
        f.setTitle("Welcome to Swings");
        f.setSize(500,500);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

JToggleButton:-

A useful variation on the push button is called a *toggle button*. A toggle button looks just like a push button, but it acts differently because it has two states: pushed and released. That is, when you press a toggle button, it stays pressed rather than popping back up as a regular push button does. When you press the toggle button a second time, it releases (pops up). Therefore, each time a toggle button is pushed, it toggles between its two states.

Toggle buttons are objects of the **JToggleButton** class. **JToggleButton** implements **AbstractButton**. In addition to creating standard toggle buttons, **JToggleButton** is a superclass for two other Swing components that also represent two-state controls. These are **JCheckBox** and **JRadioButton**, which are described later in this chapter. Thus, **JToggleButton** defines the

basic functionality of all two-state components.

JToggleButton defines several constructors. The one used by the example in this section is shown here:

```
JToggleButton(String str)
```

This creates a toggle button that contains the text passed in *str*. By default, the button is in the off position. Other constructors enable you to create toggle buttons that contain images, or images and text.

JToggleButton uses a model defined by a nested class called **JToggleButton.ToggleButtonModel**. Normally, you won't need to interact directly with the model to use a standard toggle button.

Like **JButton**, **JToggleButton** generates an action event each time it is pressed. Unlike **JButton**, however, **JToggleButton** also generates an item event. This event is used by those components that support the concept of selection. When a **JToggleButton** is pressed in, it is selected. When it is popped out, it is deselected.

To handle item events, you must implement the **ItemListener** interface. Recall from [Chapter 24](#), that each time an item event is generated, it is passed to the **itemStateChanged()** method defined by **ItemListener**. Inside **itemStateChanged()**, the **getItem()** method can be called on the **ItemEvent** object to obtain a reference to the **JToggleButton** instance that generated the event. It is shown here:

```
Object getItem( )
```

A reference to the button is returned. You will need to cast this reference to **JToggleButton**.

The easiest way to determine a toggle button's state is by calling the **isSelected()** method (inherited from **AbstractButton**) on the button that generated the event. It is shown here:

```
boolean isSelected( )
```

It returns **true** if the button is selected and **false** otherwise.

Here is an example that uses a toggle button. Notice how the item listener works. It simply calls **isSelected()** to determine the button's state.

```
// Demonstrate JToggleButton.
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class JToggleButtonDemo {

    public JToggleButtonDemo() {
        // Set up the JFrame.
        JFrame jfrm = new JFrame("JToggleButtonDemo");
        jfrm.setLayout(new FlowLayout());
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        jfrm.setSize(200, 100);

        // Create a label.
        JLabel jlab = new JLabel("Button is off.");

        // Make a toggle button.
        JToggleButton jtbn = new JToggleButton("On/Off");

        // Add an item listener for the toggle button.
        jtbn.addItemListener(new ItemListener() {
            public void itemStateChanged(ItemEvent ie) {
                if(jtbn.isSelected())
                    jlab.setText("Button is on.");
                else
                    jlab.setText("Button is off.");
            }
        });

        // Add the toggle button and label to the content pane.
        jfrm.add(jtbn);
        jfrm.add(jlab);

        // Display the frame.
        jfrm.setVisible(true);
    }

    public static void main(String[] args) {
        // Create the frame on the event dispatching thread.

        SwingUtilities.invokeLater(
            new Runnable() {
                public void run() {
                    new JToggleButtonDemo();
                }
            }
        );
    }
}
```

The output from the toggle button example is shown here:



JCheckBox:

The **JCheckBox** class, which provides the functionality of a check box, is a concrete implementation of **AbstractButton**. Its immediate super class is **JToggleButton**, which provides support for two-state buttons (true or false). Some of its constructors are shown here:

```
JCheckBox(Icon i)
JCheckBox(Icon i, boolean state)
JCheckBox(String s)
JCheckBox(String s, boolean state)
JCheckBox(String s, Icon i)
JCheckBox(String s, Icon i, boolean state)
```

Here, *i* is the icon for the button. The text is specified by *s*. If *state* is **true**, the check box is initially selected. Otherwise, it is not.

The state of the check box can be changed via the following method:

```
void setSelected(boolean state)
```

Here, *state* is **true** if the check box should be checked.

When a check box is selected or deselected, an item event is generated. This is handled by **itemStateChanged()**. Inside **itemStateChanged()**, the **getItem()** method gets the **JCheckBox** object that generated the event. The **getText()** method gets the text for that check box and uses it to set the text inside the text field.

Example:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MyFrame extends JFrame implements ItemListener
{
    JCheckBox jcb,jcb1,jcb2;
    JLabel jl;
    MyFrame()
    {
        setLayout(new FlowLayout());
        jl=new JLabel();

        jcb=new JCheckBox("JAVA");
        jcb1=new JCheckBox("DBMS");
        jcb2=new JCheckBox("C PROG" );
```

```

        add(jcb); add(jcb1); add(jcb2); add(jl);

        jcb.addItemListener(this);
        jcb1.addItemListener(this);
        jcb2.addItemListener(this);
    }
    public void itemStateChanged(ItemEvent ie)
    {
        JCheckBox jc=(JCheckBox)ie.getItem();
        jl.setText("You Selected :"+jc.getText() );
    }
}
class FrameDemo
{
    public static void main(String arg[])
    {
        MyFrame f=new MyFrame();
        f.setTitle("Welcome to Swings");
        f.setSize(500,500);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

JRadioButton:

Radio buttons are supported by the **JRadioButton** class, which is a concrete implementation of **AbstractButton**. Its immediate superclass is **JToggleButton**, which provides support for two-state buttons. Some of its constructors are shown here:

```

JRadioButton(Icon i)
JRadioButton(Icon i, boolean state)
JRadioButton(String s)
JRadioButton(String s, boolean state)
JRadioButton(String s, Icon i)
JRadioButton(String s, Icon i, boolean state)

```

Here, *i* is the icon for the button. The text is specified by *s*. If *state* is **true**, the button is initially selected. Otherwise, it is not.

Radio buttons must be configured into a group. Only one of the buttons in that group can be selected at any time. For example, if a user presses a radio button that is in a group, any previously selected button in that group is automatically deselected. The **ButtonGroup** class is instantiated to create a button group. Its default constructor is invoked for this purpose. Elements are then added to the button group via the following method:

```
void add(AbstractButton ab)
```

Here, *ab* is a reference to the button to be added to the group.

Radio button presses generate action events that are handled by **actionPerformed()**.

The **getActionCommand()** method returns the text that is associated with a radio button and uses it to set the text field.

Example:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MyFrame extends JFrame implements ActionListener
{
    JRadioButton jrb,jrb1,jrb2;
    JLabel jl;
    MyFrame()
    {
        setLayout(new FlowLayout());
        jl=new JLabel();

        jrb=new JRadioButton("VRSEC");
        jrb1=new JRadioButton("PVPSIT");
        jrb2=new JRadioButton("BEC" );

        add(jrb); add(jrb1); add(jrb2); add(jl);

        ButtonGroup bg=new ButtonGroup();
        bg.add(jrb); bg.add(jrb1); bg.add(jrb2);

        jrb.addActionListener(this);
        jrb1.addActionListener(this);
        jrb2.addActionListener(this);
    }
    public void actionPerformed(ActionEvent ae)
    {
        jl.setText("You Selected :"+ae.getActionCommand());
    }
}
class FrameDemo
{
    public static void main(String arg[])
    {
        MyFrame f=new MyFrame();
        f.setTitle("Welcome to Swings");
        f.setSize(500,500);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

```
}  
}
```

Event Handling

The preceding example showed the basic form of a Swing program, but it left out one important part: event handling. Because **JLabel** does not take input from the user, it does not generate events, so no event handling was needed. However, the other Swing components *do* respond to user input and the events generated by those interactions need to be handled. Events can also be generated in ways not directly related to user input. For example, an event is generated when a timer goes off. Whatever the case, event handling is a large part of any Swing-based application.

The event handling mechanism used by Swing is the same as that used by the AWT. This approach is called the *delegation event model*. In many cases, Swing uses the same events as does the AWT, and these events are packaged in **java.awt.event**. Events specific to Swing are stored in **javax.swing.event**.

Although events are handled in Swing in the same way as they are with the AWT, it is still useful to work through a simple example. The following program handles the event generated by a Swing push button. Sample output is shown in [Figure 31-2](#).

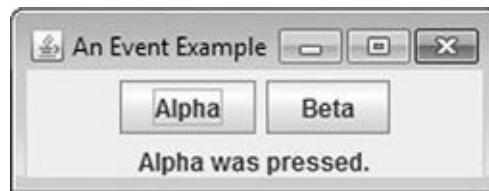


Figure 31-2 Output from the **EventDemo** program

```
// Handle an event in a Swing program.

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class EventDemo {

    JLabel jlab;

    EventDemo() {

        // Create a new JFrame container.
        JFrame jfrm = new JFrame("An Event Example");

        // Specify FlowLayout for the layout manager.
        jfrm.setLayout(new FlowLayout());

        // Give the frame an initial size.
        jfrm.setSize(220, 90);
```

```
// Terminate the program when the user closes the application.
jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

// Make two buttons.
JButton jbtnAlpha = new JButton("Alpha");
JButton jbtnBeta = new JButton("Beta");

// Add action listener for Alpha.
jbtnAlpha.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent ae) {
        jlab.setText("Alpha was pressed.");
    }
});

// Add action listener for Beta.
jbtnBeta.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent ae) {
        jlab.setText("Beta was pressed.");
    }
});

// Add the buttons to the content pane.
jfrm.add(jbtnAlpha);
jfrm.add(jbtnBeta);

// Create a text-based label.
jlab = new JLabel("Press a button.");

// Add the label to the content pane.
jfrm.add(jlab);

// Display the frame.
jfrm.setVisible(true);
}

public static void main(String args[]) {
    // Create the frame on the event dispatching thread.
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new EventDemo();
        }
    });
}
}
```

First, notice that the program now imports both the **java.awt** and **java.awt.event** packages. The **java.awt** package is needed because it contains the **FlowLayout** class, which supports the standard flow layout manager used to lay out components in a frame. (See Chapter 26 for coverage of layout managers.) The **java.awt.event** package is needed because it defines the **ActionListener** interface and the **ActionEvent** class.

The **EventDemo** constructor begins by creating a **JFrame** called **jfrm**. It then sets the layout manager for the content pane of **jfrm** to **FlowLayout**. By default, the content pane uses **BorderLayout** as its layout manager. However, for this example, **FlowLayout** is more convenient.

After setting the size and default close operation, **EventDemo()** creates two push buttons, as shown here:

```
JButton jbtnAlpha = new JButton("Alpha"); JButton jbtnBeta =  
new JButton("Beta");
```

The first button will contain the text "Alpha" and the second will contain the text "Beta". Swing push buttons are instances of **JButton**. **JButton** supplies several constructors. The one used here is

```
JButton(String msg)
```

The *msg* parameter specifies the string that will be displayed inside the button.

When a push button is pressed, it generates an **ActionEvent**. Thus, **JButton** provides the **addActionListener()** method, which is used to add an action listener. (**JButton** also provides **removeActionListener()** to remove a listener, but this method is not used by the program.) As explained in [Chapter 24](#), the **ActionListener** interface defines only one method: **actionPerformed()**. It is shown again here for your convenience:

```
void actionPerformed(ActionEvent ae)
```

This method is called when a button is pressed. In other words, it is the event handler that is called when a button press event has occurred.

Next, event listeners for the button's action events are added by the code shown here:

```
// Add action listener for Alpha.
jbtnAlpha.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent ae) {
        jlab.setText("Alpha was pressed.");
    }
});

// Add action listener for Beta.
jbtnBeta.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent ae) {
        jlab.setText("Beta was pressed.");
    }
});
```

Here, anonymous inner classes are used to provide the event handlers for the two buttons. Each time a button is pressed, the string displayed in **jlab** is changed to reflect which button was pressed.

Beginning with JDK 8, lambda expressions can also be used to implement some types of event handlers. For example, the event handler for the Alpha button could be written like this:

```
jbtnAlpha.addActionListener( (ae) -> jlab.setText("Alpha was pressed.));
```

As you can see, this code is shorter. Of course, the approach you choose will be determined by the situation and your own preferences.

Next, the buttons are added to the content pane of **jfrm**:

```
jfrm.add(jbtnAlpha); jfrm.add(jbtnBeta);
```

Finally, **jlab** is added to the content pane and the window is made visible. When you run the program, each time you press a button, a message is displayed in the label that indicates which button was pressed.

One last point: Remember that all event handlers, such as **actionPerformed()**, are called on the event dispatching thread. Therefore, an event handler must return quickly in order to avoid slowing down the application. If your application needs to do something time consuming as the result of an event, it must use a separate thread.